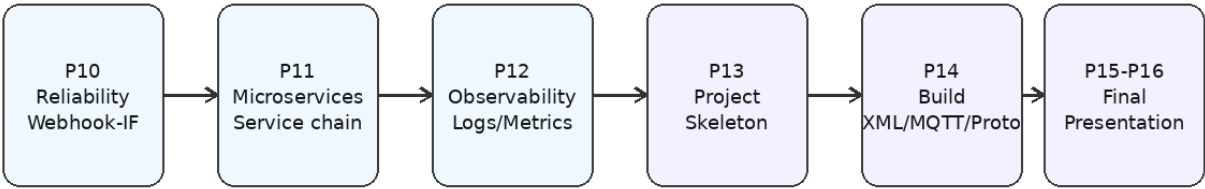


PANDUAN LAB n8n SELF-HOSTED

COMMUNICATION PROTOCOLS P10-P16

Gabungan Panduan Dosen + Lab Manual Mahasiswa
Program Studi Sains Data - Semester 2

Roadmap P10-P16: n8n grows into UAS portfolio



Rule: P15-P16 are not new-tool sessions. Students must present a working workflow, evidence, and protocol rationale.

Dokumen ini: Disusun untuk membantu delivery Pertemuan 10 sampai UAS: API reliability, n8n workflow integration, microservices, observability, XML, MQTT, Protobuf, dan final project evidence. Bahasa utama Indonesia dengan istilah teknis English sesuai materi kelas.

Item	Keterangan
Audience	Dosen dan mahasiswa semester 2 Sains Data, kelas reguler dan kelas profesional.
Mode pembelajaran	Formal akademik + lab manual step-by-step.
Environment utama	Windows + Docker Desktop; Debian-family Linux sebagai alternatif.
Tools	Python, Postman, Docker, n8n self-hosted Community, Wireshark opsional, MQTT broker lokal embedded.
Target akhir	P15-P16 final presentation berbasis mini project dengan evidence.

Daftar Isi Ringkas

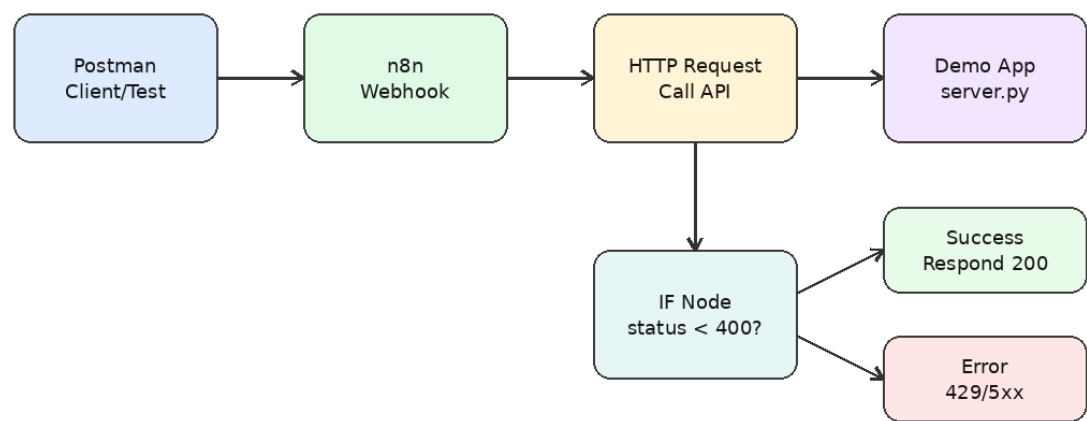
- 1. Executive summary dan prinsip delivery
- 2. Hasil inspeksi attachment dan refactor target
- 3. Mapping RPS P10-P16
- 4. Setup environment Windows dan Debian-family Linux
- 5. Konsep Docker networking: localhost vs host.docker.internal
- 6. Menjalankan demo app server.py
- 7. Postman baseline dan test script
- 8. Workflow n8n P10 step-by-step
- 9. Pengembangan P11: microservices communication
- 10. Pengembangan P12: observability dan logging
- 11. P13-P14: project skeleton dan final build
- 12. P15-P16: final presentation UAS
- 13. Parameter use case reguler dan profesional
- 14. Protobuf, XML, MQTT lokal, dan Wireshark evidence
- 15. Smoke test, troubleshooting, rubrik, dan referensi

Catatan: Jika dokumen ini dibagikan ke mahasiswa, bagian quick start, workflow P10, deliverables, troubleshooting, dan rubrik adalah bagian yang paling penting dibaca terlebih dahulu.

1. Executive Summary dan Prinsip Delivery

n8n dipakai sebagai workflow orchestration layer untuk memperlihatkan bagaimana request, API call, status code, decision branch, logging, dan evidence saling terhubung. n8n bukan pengganti konsep protocol dan bukan pengganti Postman. Postman tetap menjadi client/tester; n8n menjadi kanvas integrasi; demo app server.py menjadi backend lokal untuk pembelajaran.

P10 n8n workflow: API reliability evidence



Komponen	Peran di Kelas
Postman	Mengirim request, menjalankan test script, collection runner, dan menyimpan evidence request-response.
n8n self-hosted	Menerima Webhook, memanggil API, membuat branch success/error, mengirim response final, dan menunjukkan execution evidence.
Demo app server.py	Backend lokal untuk REST, XML, MQTT, Protobuf, reliability, observability, automation, dan UAS capture demo.
Wireshark	Opsional untuk evidence protocol packet: HTTP, MQTT, WebSocket, gRPC HTTP/2, TLS.

Prinsip delivery: Mulai dari konsep, lanjutkan demo dosen, baru guided lab mahasiswa. Jangan langsung meminta mahasiswa membuat workflow kompleks di P10.

2. Hasil Inspeksi Attachment dan Refactor Target

Attachment yang diinspeksi berisi demo app Python dengan satu entripoint utama server.py. Aplikasi sudah mencakup endpoint P9-P16, local HTTP API di port 8088, local MQTT broker di port 1883, gRPC HTTP/2 di port 50051, TLS demo di port 8443, Postman collection, dan lab markdown.

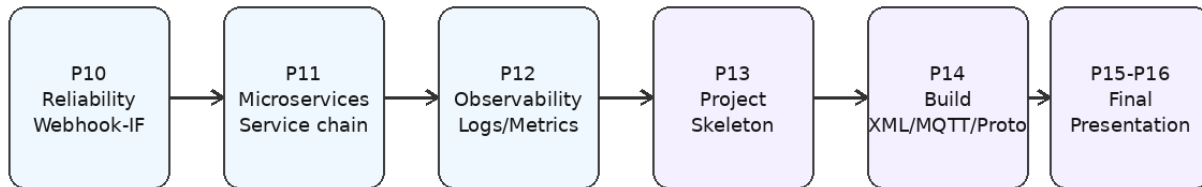
Area	Hasil inspeksi	Rekomendasi refactor
Struktur runtime	server.py sudah menjadi satu	Pertahankan satu entripoint

	runtime utama.	server.py agar mahasiswa cukup menjalankan satu program.
Dependency	requirements.txt berisi grpcio dan protobuf.	Tetapkan Python latest stable yang tersedia di laptop; gunakan pip install -r requirements.txt.
Data/static	Ada folder data dan static.	Boleh tetap disertakan; untuk single-entrypoint, server.py tetap menjadi proses utama.
Protobuf	Versi awal encoding Protobuf pada /api/encoding/convert masih berupa base64 demo dari JSON.	Refactor dilakukan agar endpoint Protobuf memakai Python protobuf library dan menghasilkan binary protobuf yang ditampilkan sebagai base64.
n8n	Ada endpoint /api/automation/* dan /api/project/capture-demo.	Cukup untuk P13-P16, ditambah workflow n8n P10 sebagai starter.

Keputusan: Target refactor bukan memecah program, tetapi menegaskan server.py sebagai satu aplikasi terpadu untuk P10 sampai UAS. Program tetap bisa menyertakan data, static, postman, dan docs sebagai artefak pendukung.

3. Mapping RPS P10-P16

Roadmap P10-P16: n8n grows into UAS portfolio



Rule: P15-P16 are not new-tool sessions. Students must present a working workflow, evidence, and protocol rationale.

Pertemuan	Topik operasional	Output mahasiswa
P10	API Reliability, Rate Limiting, Load Balancing, intro n8n workflow.	Workflow Webhook -> HTTP Request -> IF -> Respond; evidence 200 dan 429.
P11	Microservices Communication Patterns.	Diagram service chain, protocol mapping, failure point.
P12	Monitoring, Logging, Telemetry, Observability.	Logs, metrics, trace, execution history, dan analisis troubleshooting.
P13	Project Protocol Phase 1.	Use case, architecture diagram, workflow skeleton, testing plan.
P14	Project Protocol Phase 2.	Final workflow: REST, XML, MQTT, Protobuf, logging, error handling.
P15	Presentation simulation and peer review.	Dry-run 10 menit, checklist revisi, evidence gap list.
P16	UAS Final Project Presentation.	Final demo, report PDF, PPT, repository, Postman collection, n8n export, evidence.

Target pedagogis: P15-P16 bukan sesi belajar tools baru. Mahasiswa harus sudah mengenal n8n sejak P10 dan mengembangkannya bertahap sampai final project.

4. Quick Start untuk Dosen

Bagian ini adalah checklist cepat sebelum kelas. Jalankan demo app terlebih dahulu, cek endpoint, lalu jalankan n8n self-hosted. Untuk kelas pertama, dosen sebaiknya melakukan demo penuh sebelum mahasiswa mencoba sendiri.

```
# Terminal 1 - demo app
cd app
```

Communication Protocols - n8n P10 sampai UAS | Universitas Cakrawala | Haikal Shiddiq S.Kom., M.T.

```
py -m pip install -r requirements.txt  
py server.py
```

```
# Cek dari browser/Postman  
http://localhost:8088/api/health
```

```
# Terminal 2 - n8n self-hosted via Docker Desktop Windows  
cd scripts  
.\run_n8n_windows.ps1
```

```
# Buka n8n  
http://localhost:5678
```

Jika memakai Debian-family Linux: Gunakan scripts/run_n8n_debian.sh. Command Linux menyertakan --add-host=host.docker.internal:host-gateway agar container bisa memanggil service di host.

5. Setup Environment Windows dan Debian-family Linux

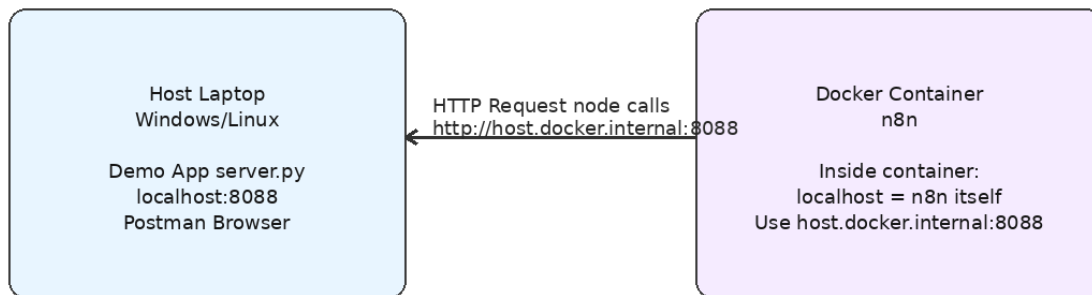
Komponen	Windows	Debian-family Linux
Python	Install Python terbaru dari python.org, centang Add to PATH. Gunakan py server.py.	Install python3 dan python3-pip dari package manager. Gunakan python3 server.py.
Docker	Docker Desktop. Pastikan WSL2 backend aktif.	Docker Engine atau Docker Desktop for Linux. Pastikan user punya akses docker.
Postman	Sudah diminta pada pertemuan sebelumnya. Import collection dari folder postman.	Sama. Bisa gunakan Postman desktop atau alternatif curl untuk beberapa test.
n8n	Run container dari command final. UI di http://localhost:5678 .	Run container dengan --add-host. UI di http://localhost:5678 .
Wireshark	Opsional untuk capture HTTP/MQTT.	Opsional; perlu privilege capture interface.

```
# Windows PowerShell - n8n final command
docker volume create n8n_data
docker run -it --rm `
  --name n8n `
  -p 5678:5678 `
  -e GENERIC_TIMEZONE="Asia/Jakarta" `
  -e TZ="Asia/Jakarta" `
  -e N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true `
  -e N8N_RUNNERS_ENABLED=true `
  -v n8n_data:/home/node/.n8n `
  docker.n8n.io/n8nio/n8n
```

```
# Debian-family Linux - n8n final command
docker volume create n8n_data
docker run -it --rm \
  --name n8n \
  -p 5678:5678 \
  --add-host=host.docker.internal:host-gateway \
  -e GENERIC_TIMEZONE="Asia/Jakarta" \
  -e TZ="Asia/Jakarta" \
  -e N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true \
  -e N8N_RUNNERS_ENABLED=true \
  -v n8n_data:/home/node/.n8n \
  docker.n8n.io/n8nio/n8n
```

6. Docker Networking: localhost vs host.docker.internal

Docker networking: localhost vs host.docker.internal



From browser/Postman on host: use `http://localhost:8088`. From n8n container: use `http://host.docker.internal:8088`.

Jika n8n berjalan di Docker, node HTTP Request di dalam n8n berada di dalam container. Karena itu, localhost di dalam node HTTP Request menunjuk ke container n8n itu sendiri, bukan laptop host. Agar n8n container bisa memanggil demo app yang berjalan di laptop pada port 8088, gunakan `host.docker.internal:8088`.

Posisi akses	URL yang digunakan
Browser/Postman di laptop	<code>http://localhost:8088</code>
n8n HTTP Request node di Docker Desktop Windows/Mac	<code>http://host.docker.internal:8088</code>
n8n HTTP Request node di Debian-family Linux	<code>http://host.docker.internal:8088</code> dengan <code>--add-host=host.docker.internal:host-gateway</code>

Kesalahan umum: Jika HTTP Request node memakai `http://localhost:8088` saat n8n berjalan di Docker, biasanya akan gagal connect karena port 8088 tidak berjalan di container n8n.

7. Menjalankan Demo App server.py

Demo app adalah local backend untuk P9-P16. Satu proses server.py akan mengaktifkan HTTP UI/API, local MQTT broker, gRPC HTTP/2 server, dan TLS demo. Setelah server berjalan, endpoint utama bisa diuji dari Postman.

```
# Windows
cd app
py -m pip install -r requirements.txt
py server.py

# Linux
cd app
python3 -m pip install -r requirements.txt
python3 server.py
```

Service	Port	Contoh endpoint
HTTP UI/API	8088	GET /api/health, GET /api/openapi
MQTT broker lokal	1883	POST /api/mqtt/real-publish sebagai trigger
gRPC HTTP/2	50051	POST /api/grpc/http2-call
TLS/HTTPS demo	8443	POST /api/tls/self-test

```
# Endpoint smoke test manual
GET http://localhost:8088/api/health
GET http://localhost:8088/api/openapi
POST http://localhost:8088/api/reliability/reset
GET http://localhost:8088/api/reliability/rate-limited
POST http://localhost:8088/api/project/capture-demo
```

8. Postman Baseline dan Test Script

Postman tetap digunakan sebagai client dan evidence generator. Untuk P9, mahasiswa harus memahami collection, environment variable, request, response, status code, dan Test Results. Untuk P10, Postman digunakan untuk trigger Webhook n8n dan validasi 200/429.

```
// P9 /api/openapi test script
pm.test("Status code 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Response is JSON", function () {
  pm.response.to.be.json;
});

const jsonData = pm.response.json();

pm.test("Has openapi field", function () {
  pm.expect(jsonData).to.have.property("openapi");
});
```

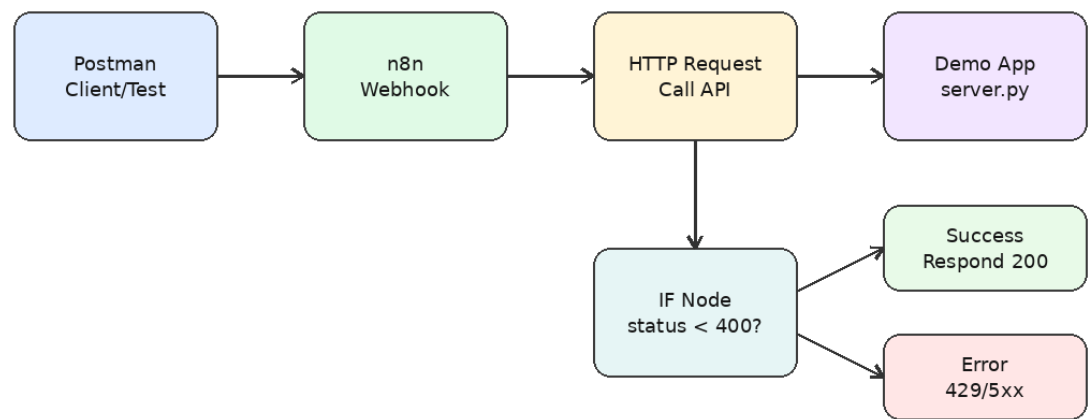
```
pm.test("Has info.title", function () {  
  pm.expect(jsonData.info).to.have.property("title");  
});  
  
pm.test("Has paths", function () {  
  pm.expect(jsonData).to.have.property("paths");  
});  
  
// P10 rate limit test script  
pm.test("Status must be 200 or 429", function () {  
  pm.expect([200, 429]).to.include(pm.response.code);  
});  
  
const jsonData = pm.response.json();  
  
if (pm.response.code === 429) {  
  pm.test("429 has retryAfterSeconds", function () {  
    pm.expect(jsonData).to.have.property("retryAfterSeconds");  
  });  
}
```

Evidence: Screenshot response saja belum cukup. Mahasiswa harus mengambil screenshot Test Results agar validasi objektif terlihat.

9. Workflow n8n P10 dari Nol

Workflow inti P10 adalah Webhook -> Edit Fields -> HTTP Request -> IF -> Respond to Webhook. Workflow ini membuktikan API reliability dengan cara membaca status code dan memisahkan success/error branch.

P10 n8n workflow: API reliability evidence



Evidence: request body, statusCode, response body, active branch, execution screenshot.

Node	Konfigurasi inti	Tujuan
Webhook	Method POST, Path commproto-p10, Response mode: Respond to Webhook node.	Menerima request dari Postman.
Edit Fields	clientId, scenario, topic, rpsMapping.	Merapikan payload agar mudah dibaca.
HTTP Request	GET http://host.docker.internal:8088/api/r eliability/rate-limited.	Memanggil backend reliability API.
IF	statusCode < 400.	Memisahkan success dan error/rate-limit branch.
Respond to Webhook	Return JSON response ke Postman.	Memberi hasil akhir workflow kepada client.

```
# Body Postman untuk Webhook n8n
{
  "clientId": "kelompok-01",
  "scenario": "rate-limit",
  "timestamp": "2026-06-18T20:00:00+07:00"
}
```

Rule P10: Jangan langsung membuat workflow kompleks. Target P10 adalah mahasiswa paham input, process, API call, status code, branch, response, dan evidence.

10. Langkah Detail Membuat Node n8n

Step 1 - Create Workflow

1. Buka <http://localhost:5678>.
2. Create Workflow.
3. Rename menjadi P10 - API Reliability Workflow.

Step 2 - Webhook Node

4. Add first step -> Webhook.
5. HTTP Method = POST.
6. Path = commproto-p10.
7. Respond = Using Respond to Webhook Node.
8. Klik Listen for test event saat testing.

Step 3 - Edit Fields Node

9. Tambahkan node Edit Fields.
10. Buat field clientId, scenario, topic, rpsMapping.
11. Gunakan expression dari body Webhook: `$json.body.clientId`.

Step 4 - HTTP Request Node

12. Tambahkan node HTTP Request.
13. Method = GET.
14. URL = <http://host.docker.internal:8088/api/reliability/rate-limited>.
15. Aktifkan Include Response Headers and Status dan Never Error.

Step 5 - IF Node

16. Tambahkan node IF.
17. Condition: statusCode smaller than 400.
18. True branch untuk success; False branch untuk 429/5xx.

Step 6 - Respond Nodes

19. True -> Build Success Response -> Respond Success.
20. False -> Build Error Response -> Respond Error.
21. Untuk kelas, response n8n boleh tetap 200 dengan body berisi backendStatusCode.

Screenshot Placeholder

Masukkan screenshot aktual dari laptop mahasiswa/dosen di sini:

- n8n workflow canvas
- Webhook input/output
- HTTP Request node output
- IF branch success/error
- Postman response / Test Results

11. Evidence P10 yang Wajib Dikumpulkan

Evidence	Kenapa wajib	Contoh interpretasi
Workflow canvas	Membuktikan node tersusun logis.	Webhook -> HTTP Request -> IF -> Respond.
Webhook input	Membuktikan request dari Postman masuk ke n8n.	Payload berisi clientId dan scenario.
HTTP Request output	Membuktikan n8n memanggil backend API.	statusCode 200 atau 429 terlihat.
IF branch aktif	Membuktikan decision logic bekerja.	Status 429 masuk error branch.
Postman response akhir	Membuktikan client menerima hasil workflow.	Body menjelaskan SUCCESS atau RETRY_NEEDED.
Analisis 1 paragraf	Membuktikan mahasiswa memahami evidence.	429 adalah rate limiting, bukan server down.

Template analisis: Pada request awal, backend mengembalikan 200 sehingga workflow masuk success branch. Setelah request dikirim cepat beberapa kali, backend mengembalikan 429 Too Many Requests sehingga workflow masuk error branch. Ini membuktikan rate limiting sebagai reliability protection.

12. Pengembangan P11: Microservices Communication

Setelah P10, workflow diperluas dari satu API call menjadi beberapa service call. Tujuannya agar mahasiswa paham service-to-service communication, failure point, dan protocol choice.

Webhook

- > Switch by scenario
- > HTTP Request: /api/microservices/orders
- > HTTP Request: /api/observability/custom-log
- > Respond to Webhook

Komponen microservices	Protocol	Evidence
API Gateway	REST/HTTP	Request body dan status code.
Order Service	REST internal simulation	Order ID dan status.
Inventory/Payment Service	Service chain simulation	Trace/failure point.
Notification/Telemetry	MQTT or event	Topic dan payload.

Endpoint P11: Gunakan GET /api/microservices/order-flow untuk konsep dan POST /api/microservices/orders untuk skenario order flow.

13. Pengembangan P12: Observability dan Logging

Observability menjawab apa yang terjadi, seberapa sering terjadi, dan request melewati komponen apa saja. Pada tahap ini mahasiswa perlu membedakan logs, metrics, dan traces.

Evidence observability	Endpoint	Pertanyaan yang dijawab
Metrics	GET /api/observability/metrics	Berapa banyak request, error, atau

		latency?
Logs	GET /api/observability/logs	Event apa yang terjadi?
Trace	GET /api/observability/trace/{id}	Request melewati komponen mana saja?
Capture scenario	POST /api/observability/capture-scenario	Apa bukti HTTP dan MQTT dalam satu skenario?

n8n P12 extension idea

Webhook

- > Normalize Request
- > HTTP Request to backend
- > Edit Fields: build log object
- > HTTP Request POST /api/observability/custom-log
- > Respond to Webhook

14. P13-P14: Project Skeleton dan Final Build

P13 adalah fase skeleton, bukan final. P14 adalah fase final build dan dokumentasi. Mahasiswa reguler bekerja kelompok 4 orang; kelas profesional dikerjakan individu.

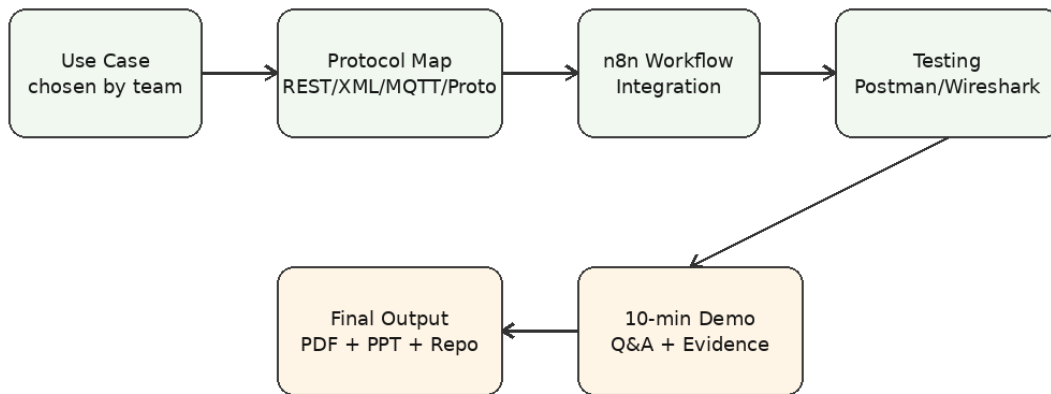
Fase	Reguler	Profesional
P13 skeleton	Kelompok 4 orang: pilih use case, buat diagram, tentukan protocol map.	Individu: buat use case lebih spesifik dan testing plan mandiri.
P14 final build	Workflow minimal REST + n8n + salah satu XML/MQTT/Protobuf tambahan.	Workflow wajib mencakup REST, XML, MQTT, Protobuf, error handling, dan observability.
Evidence	Screenshot dan analisis kelompok.	Export workflow JSON, Postman collection, log, dan analisis lebih mendalam.

Final workflow pattern

Input REST Webhook

- > Validate payload
- > Convert XML or Protobuf if needed
- > Publish/trigger MQTT telemetry
- > Call backend API
- > Log result
- > Return response

UAS evidence pipeline



Minimum: success case, error case, logs, status codes, protocol explanation, and exportable artifacts.

15. P15-P16: Final Presentation UAS

Durasi presentasi maksimal 10 menit. Karena waktu sempit, mahasiswa harus langsung menunjukkan use case, architecture, workflow, demo, evidence, dan kesimpulan. Tidak perlu menjelaskan tools dari nol.

Menit	Isi presentasi
0:00-1:00	Problem statement dan use case.
1:00-2:00	Architecture diagram dan protocol mapping.
2:00-5:00	Demo n8n workflow dan Postman trigger.
5:00-7:00	Evidence success, error, log/observability, status code.
7:00-8:30	Reliability/troubleshooting analysis.
8:30-10:00	Kesimpulan dan Q&A cepat.

Artefak final	Reguler	Profesional
PDF report	Wajib kelompok.	Wajib individu.
PPT final	Wajib.	Wajib.
Repository	Wajib, boleh satu repo kelompok.	Wajib personal.
Postman Collection	Wajib.	Wajib dengan test script.
n8n workflow export	Wajib.	Wajib dan harus bisa dijelaskan per node.
Evidence logs/metrics	Minimal screenshot.	Screenshot + analisis root cause/failure.

16. Parameter Use Case Final

Use case boleh bebas, tetapi harus memenuhi parameter teknis minimal. Tujuannya agar project tidak hanya menjadi automation sederhana, tetapi tetap terikat ke Communication Protocols.

Parameter	Reguler - kelompok 4 orang	Profesional - individu
Protocol coverage	Minimal REST+JSON, n8n, dan dua dari XML/MQTT/Protobuf.	Wajib REST+JSON, XML, MQTT lokal, Protobuf library, dan n8n.
Reliability	Ada success case dan error case.	Ada success, error, retry/fallback note, dan log.
Observability	Screenshot execution dan endpoint logs.	Execution history, custom log, metrics/trace evidence.
Documentation	Diagram + laporan + collection.	Diagram + laporan rinci + exported workflow + test scripts.
Presentation	Maksimal 10 menit kelompok.	Maksimal 10 menit individu.

Contoh use case	Keterangan
Smart Campus Telemetry Alert	Sensor suhu/kelembaban mengirim data, n8n memvalidasi, API mencatat, error branch dibuat.
Library Occupancy Monitoring	Data okupansi ruangan dikirim via REST/MQTT dan dicatat untuk observability.
Mini E-Commerce Reliability Monitor	Order API diuji status code, load balancing, dan logging.
Lab Equipment Health Check	Data alat lab dikirim, XML/Protobuf dipakai untuk format exchange.

17. XML, MQTT Lokal, dan Protobuf Library

Mahasiswa sudah cukup paham REST API dan JSON. Karena itu, P10 ke atas harus menjadi jembatan ke XML, MQTT, dan Protobuf melalui use case yang sama agar tidak terasa sebagai materi terpisah.

Format/protocol	Endpoint demo	Teaching point
XML	POST /api/encoding/convert { "to": "xml" }	Legacy/enterprise exchange; cocok untuk sistem lama dan interoperability.
MQTT lokal	POST /api/mqtt/real-publish	Publish-subscribe telemetry; local broker berjalan di server.py pada port 1883.
Protobuf	POST /api/encoding/convert { "to": "protobuf" }	Schema-based binary encoding; hasil binary ditampilkan sebagai base64 agar aman dibaca di Postman.

```
# Protobuf conversion body
{
  "to": "protobuf",
  "records": [
    { "id": 1, "sensor": "temperature", "value": 31.2, "unit": "celsius" }
  ]
}
```

Implementasi refactor: Endpoint Protobuf pada paket ini memakai Python protobuf library. Output binary protobuf dikembalikan sebagai encodedBase64 dan decodedJson untuk memudahkan pembelajaran.

18. Wireshark Evidence Opsional

Protocol	Trigger	Display filter
HTTP	GET /api/health atau Postman ke Webhook/API	tcp.port == 8088 atau http
MQTT	POST /api/mqtt/real-publish	tcp.port == 1883 atau mqtt
WebSocket	POST /api/websocket/real-send	http.request.uri contains "/ws/echo" atau websocket
gRPC HTTP/2	POST /api/grpc/http2-call	tcp.port == 50051 atau http2
TLS	POST /api/tls/self-test	tcp.port == 8443 atau tls

Batasan: Wireshark evidence opsional untuk kelas reguler jika waktu terbatas. Untuk kelas profesional, minimal satu capture HTTP atau MQTT disarankan.

19. Smoke Test dan Validasi

Smoke test lokal sudah dilakukan pada demo app refactored. n8n Docker command disusun berdasarkan dokumentasi resmi n8n dan Docker; Docker daemon tidak dijalankan di sandbox ini, sehingga validasi Docker dilakukan dengan command yang perlu dijalankan pada laptop dosen/mahasiswa.

Area	Hasil
Demo app server.py	PASS: /api/health, /api/openapi, reliability, microservices, observability, automation, project, XML, Protobuf, MQTT.
Rate limiting	PASS: request keempat mengembalikan 429 setelah limit.
Load balancing	PASS: backend berubah api-node-a/b/c pada request berulang.
Protobuf library	PASS: binary protobuf dihasilkan dan dikembalikan sebagai base64.
MQTT lokal	PASS: /api/mqtt/real-publish mengembalikan packet evidence.
n8n Docker	Command final disediakan untuk Windows dan Debian-family Linux; perlu run test pada laptop dengan Docker Desktop/Engine.

```
# Test dari dalam container n8n jika perlu
docker exec -it n8n sh
wget -qO- http://host.docker.internal:8088/api/health
```

```
# Jika wget tidak ada
node -e "fetch('http://host.docker.internal:8088/api/health').then(r=>r.text()).then(console.log).catch(console.error)"
```

20. Troubleshooting

Masalah	Probable cause	Perbaikan
n8n HTTP Request gagal connect	URL masih localhost dari dalam container.	Ganti menjadi host.docker.internal:8088. Di Linux tambahkan --add-host.
Webhook tidak menerima request	Belum klik Listen for test event atau method salah.	Klik Listen for test event; pastikan Postman POST dan URL Test webhook.
429 tidak muncul	Request tidak cukup cepat atau state belum reset.	POST /api/reliability/reset lalu hit 4-5 kali cepat.
IF selalu masuk success	statusCode tidak terbaca.	Aktifkan Include Response Headers and Status dan Never Error pada HTTP Request.
Postman Test gagal	Field response tidak sesuai script.	Baca response aktual lalu sesuaikan property yang dicek.
Port 1883/50051/8443 bentrok	Service lain sudah memakai port.	Tutup service lain atau fokus pada HTTP 8088 untuk demo awal.

21. Rubrik Penilaian P10 sampai UAS

Komponen	Bobot rekomendasi	Indikator
Protocol design and rationale	20%	Protocol dipilih sesuai kebutuhan, bukan sekadar disebut.
Implementation/demo	25%	Workflow berjalan, API bisa dipanggil, error branch ada.
Testing and evidence	20%	Postman Test Results, status code, execution screenshot, success/error case.
n8n integration evidence	15%	Webhook, HTTP Request, IF, Respond, dan export workflow.
Observability and troubleshooting	10%	Logs/metrics/trace atau execution history dan analisis failure.
Documentation and presentation	10%	PDF/PPT/repository rapi dan presentasi maksimal 10 menit.

22. Appendix - Command dan Endpoint Penting

App commands

```
py -m pip install -r requirements.txt
```

```
py server.py
```

Important URLs

```
http://localhost:8088/api/health
```

```
http://localhost:8088/api/openapi
```

```
http://localhost:8088/api/reliability/rate-limited
```

```
http://localhost:8088/api/reliability/load-balanced
```

```
http://localhost:8088/api/automation/n8n-workflow-spec
```

```
http://localhost:8088/api/project/capture-demo
```

n8n UI

`http://localhost:5678`

URL inside n8n Docker to call app on host
`http://host.docker.internal:8088/api/health`

23. Referensi Resmi

- **n8n Docker installation:** <https://docs.n8n.io/hosting/installation/docker/>
- **n8n Webhook node:** <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/>
- **n8n HTTP Request node:** <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/>
- **n8n Respond to Webhook node:** <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.respondtowehook/>
- **Docker Desktop networking and host services:**
<https://docs.docker.com/desktop/features/networking/networking-how-tos/>
- **Protocol Buffers Python basics:** <https://protobuf.dev/getting-started/pythontutorial/>
- **Protocol Buffers Python reference:** <https://protobuf.dev/reference/python/>

Penutup: Dokumen ini dirancang agar n8n dapat dipakai bertahap dari P10 sampai P16. Mulai dari reliability workflow sederhana, lalu tambah microservices, observability, XML, MQTT, Protobuf, dan final project evidence.